

Detecting Card Transactions Fraud with Machine Learning

프로젝트 수행 기간 : 2024.09 ~ 2024.10

역할	이름	학번
팀장	장○곤	2020xxxx
팀원	이수아	2021xxxx
팀원	이○진	2021xxxx
팀원	임○현	2020xxxx

1 INTRODUCTION

현대 사회에서 신용카드 사용의 증가와 함께 카드 사기로 인한 금융 피해가 급증하고 있다. 이에 따라 신용카드 거래 내역을 분석하여 사기성 거래를 탐지하는 시스템의 필요성이 강조되고 있다. 본 프로젝트의 목적은 카드 사기로 인한 금융적 손실과 탐지 시스템의 운영 비용을 최소화할 수 있는 효과적인 모델을 개발하는 데 있다.

본 연구에서는 실제 카드 거래 데이터를 분석하여 사기성 거래를 탐지하는 여러 머신러닝 기법을 제안하고, 그 성능을 검증한다. 특히 선형 분류 모델, PCA(주성분 분석), EM 군집화 알고리즘 등을 통해 데이터의 차원을 축소하거나 특징을 선택하여 모델의 효율성을 높이하고자 한다. 또한, Logistic regression, Random Forest와 같은 모델을 활용해 사기 거래 탐지 성능을 비교하고, 최적의 모델을 선정한다.

2 PRE-PROCESSING

이 프로젝트의 데이터셋은 카드 거래 정보와 사용자 정보를 포함하며, 주요 목적은 사기 탐지를 위한 머신러닝 모델을 개발하는 데 사용된다.

데이터셋은 총 30개의 컬럼으로 구성되어 있으며, 사용자 정보, 카드 정보, 거래 정보 등 다양한 변수를 포함한다. 각 변수는 카드 거래와 관련된 중요한 특징을 나타낸다.

- **사용자 정보:** 사용자 식별자, 성별, 현재 나이, 은퇴 나이, 출생 연도 및 월, 우편번호, 1인당 소득, 연간 소득, 총 부채, 신용 점수
- **카드 정보:** 카드 식별자, 카드 브랜드, 카드 유형, 카드 번호, 만료일, 보안 칩 탑재 여부, 신용 한도, 계좌 개설일, 비밀번호 마지막 변경 연도
- **거래 정보:** 거래 연도, 월, 일, 거래 금액, 상품 코드

- **보안 및 사기 관련 정보:** 보안 칩 사용 여부, 오류 메시지, 사기 여부(정답 레이블)

데이터의 주요 변수는 사용자, 카드, 거래, 보안 정보 등이 있다. 분석 단계에서는 각 변수 간 상관관계를 분석했다. 분석에 앞서 범주형 데이터들을 확인한 이후, 날짜 데이터들을 연도와 월을 따로 추출하여 수치형 데이터로 변경했다.

```
# 'Expires' 칼럼에서 연도와 월 추출
data['Expires_Year'] = data['Expires'].dt.year
data['Expires_Month'] = data['Expires'].dt.month

# 'Acct Open Date' 칼럼에서 연도와 월 추출
data['AcctOpen_Year'] = data['Acct Open Date'].dt.year
data['AcctOpen_Month'] = data['Acct Open Date'].dt.month

data = data.drop(columns=['Expires', 'Acct Open Date'])
```

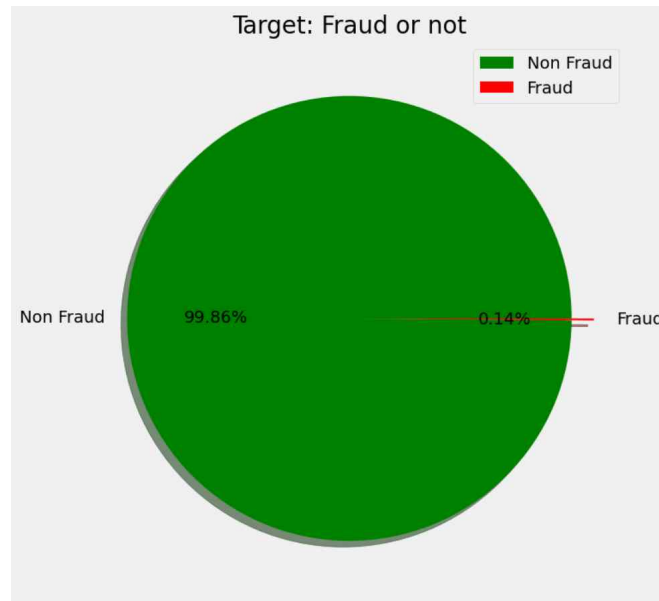
또, 결측값이 존재할 경우 모델의 성능이 저하되거나 오류가 발생할 수 있기 때문에 이를 적절히 처리했다. 칼럼별 결측값의 비율을 구한 후, 비율=1 인 칼럼을 제거하는 방식으로 제거를 진행했다. 위 결과 Error Message 칼럼이 의미 없는 값이기 때문에 제거하였으며, 그 외 칼럼에서는 결측값이 관측되지 않았으므로 추가로 결측값 처리는 하지 않았다.

위 결측값 처리까지 마친 이후 변수 간의 상관성을 관측하고 이에 따른 데이터 정제를 시도했다. 상관계수가 높은 변수를 제거하여 보다 간결하고 유용한 Feature만을 남기는 작업을 진행하였다.

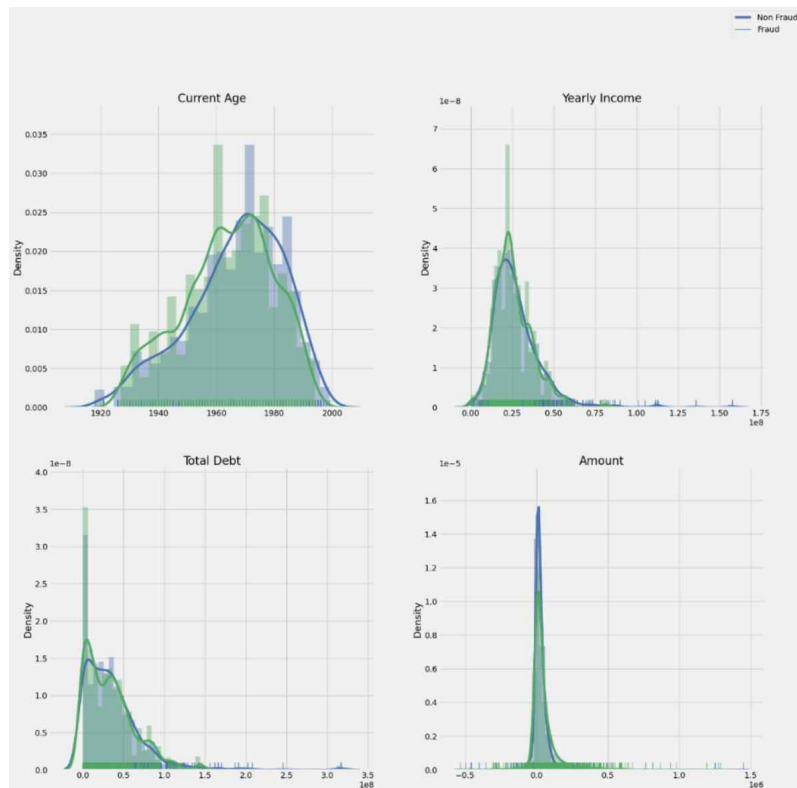
우선 값이 1개인 경우나 모든 행의 값이 다를 경우 상관성 분석에 의미가 없어 찾아 제거하였다. Year는 하나만 있으므로 제거하였고, 'Gender'와 같은 범주형 칼럼은 사실 object이나, number로 표현되어 있으므로 유지시켰으며, User, Card는 1509명에 대한 고유 ID 이므로 제거했다.

이후 Pearson 상관계수(Pearson's R)를 사용하여 숫자형 변수들 간의 상관관계를 측정하였다. Pearson 상관계수는 두 변수 간의 선형 관계를 측정하는 지표로, 상관계수가 0.9 이상인 변수는 문제를 일으킬 수 있으므로 상관도가 0.9를 넘는 'Current Age'를 제거하였다. 다음으로 범주형 데이터들의 상관도에 따른 칼럼을 확인하였다. 범주형 데이터 상관도에는 특이사항이 존재하지 않아 지우지 않았고, 나머지는 One Hot Encoding을 통해 수치형으로 변경하였다.

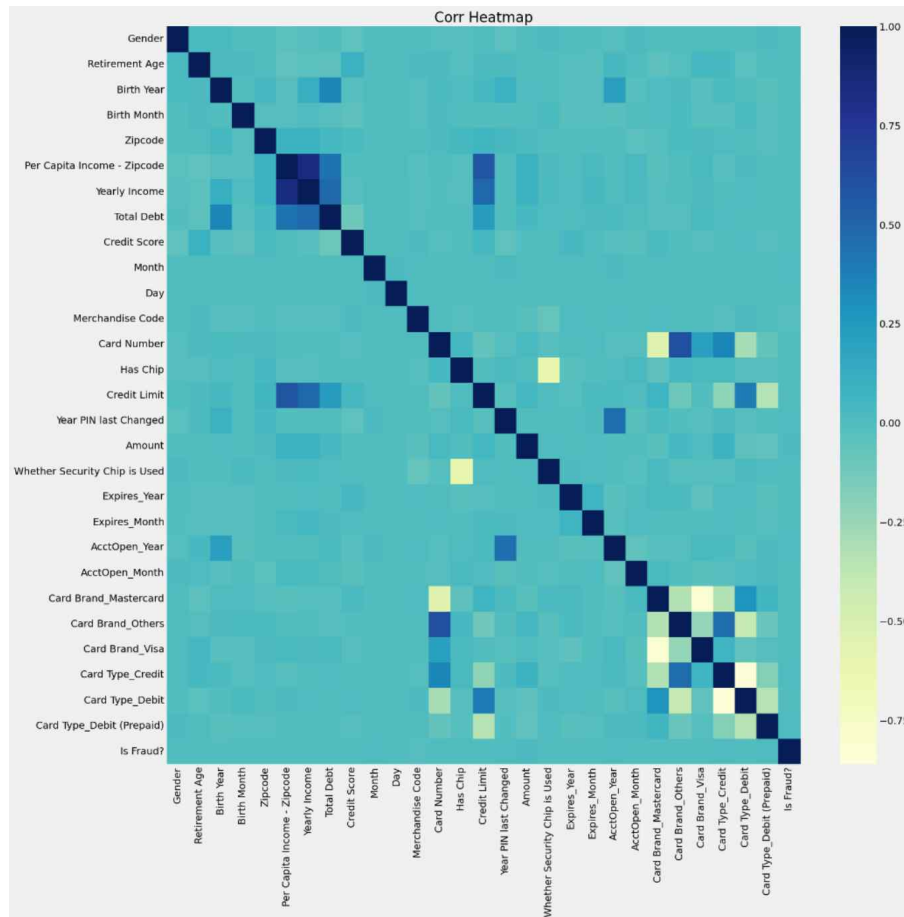
다음 Pie Chart를 통해 굉장히 불균형한 데이터를 확인할 수 있었으며, 정상 거래 대비 사기 건수가 크지 않다는 것을 확인할 수 있었다.



그렇다면 임의의 관련이 있을 법한 Feature 4개를 뽑아 Target Feature인 Fraud분포가 어떻게 되어있는지 그래프로 확인해보았다.



이 그래프들은 주요 변수를 기준으로 사기와 비사기 거래 간의 분포 차이를 시각화한 것이다. 대체적으로 각 변수의 분포는 두 집단 간 큰 차이가 없거나 매우 유사한 경향을 보인다. 따라서, 나이, 소득, 부채, 거래 금액 같은 변수들은 단독으로 사기를 예측하는 데 유용하지 않을 수 있으며, 다른 변수와의 조합 또는 상호작용을 고려한 추가 분석이 필요할 것으로 보인다.



위 히트맵은 각 Feature별 상관도를 히트맵으로 시각화하여 상관성을 나타낸 것이다. 색이 진할수록 상관계수가 높아 의미가 없다는 것을 의미한다. 여기서 임계값을 70%로 두어 그 이상인 칼럼들은 삭제했다. 히트맵의 내용을 토대로 입력한 상관계수 threshold에 따라 Feature 들 필터링하는 함수 정의 상관 계수가 높으면 제거했다.

모든 내용을 진행한 결과, 최종적으로 Feature가 26개가 남았다.

3 EXPERIMENTAL RESULTS

진행하기 이전 3가지 모델을 사용하기로 결정했다.

- XGBoost

XGBoost(Extreme Gradient Boosting)는 이전 모델의 오류를 순차적으로 보완해 나가는 방식으로 모델을 형성하는데, 더 자세히 알아보자면, 이전 모델에서의 실제값과 예측값의 오차(loss)를 훈련 데이터로 투입하고 gradient를 이용하여 오류를 보완하는 방식을 사용한다. 모든 샘플에 대해 평균값이나 최빈값으로 예측하는 초기 모델을 설정한다. 초기 모델의 예측값과 실제 값의 차이를 계산한다. 잔여 오차를 줄이기 위해 새로운 결정 트리를 학습한다. 이때 각 노드는 오차를 줄이는 방향으로 최적의 조건을 찾는다. 새로운 트리를 기존 모델에 추가하여 오차를 점진적으로 줄인다. 정해진 수의 트리 또는 조기 종료 조건에 도달할 때까지 위 과정을 반복한다.

- Random Forest

Random Forest는 다수의 결정 트리(Decision Tree)를 결합하여 최종 예측 결과를 도출하는 알고리즘이다. 여러 개의 부트스트랩 샘플(중복을 허용한 무작위 샘플)을 생성한다. 각 샘플은 서로 다른 결정 트리를 학습하는 데 사용된다.

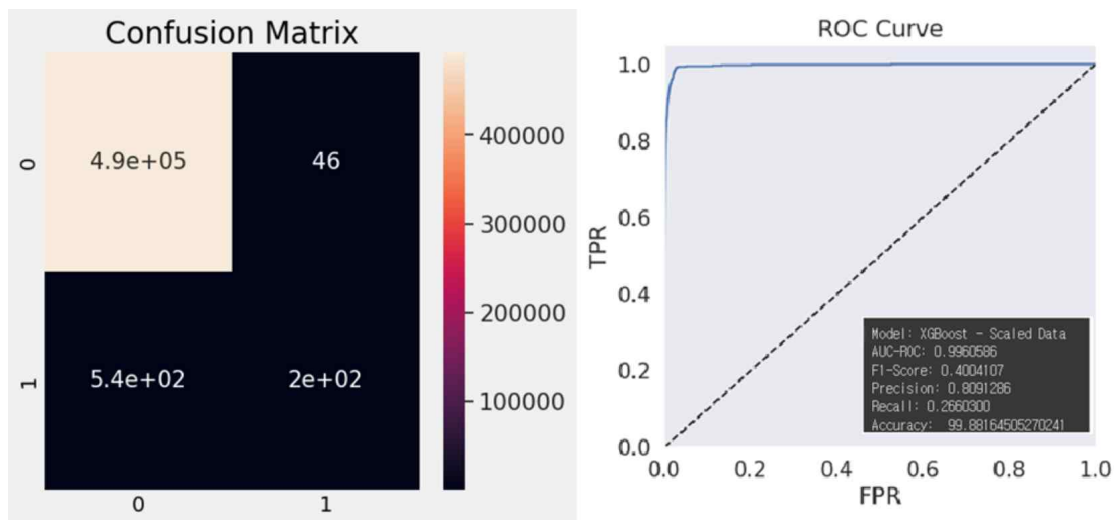
- Logistic Regression

데이터가 어떤 범주에 속할 확률을 0에서 1 사이의 값으로 예측하고 그 확률에 따라 가능성이 더 높은 범주에 속하는 것으로 분류해 주는 지도 학습 알고리즘이다.

각 결과의 AUC-ROC와 Accuracy, confusion_matrix의 결과는 AUC-ROC의 x축이 FPR(False Positive Rate), y축이 TPR(True Positive Rate)이며, confusion_matrix의 y축이 actual value, x축이 prediction이다.

1) 초기 Raw Data로 진행한 결과

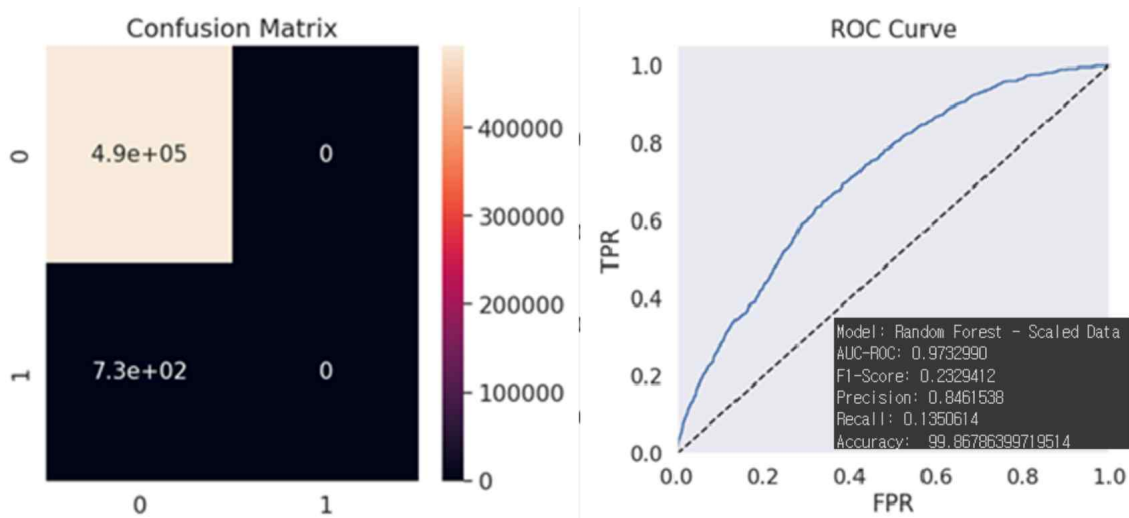
- XGBoost 사용 결과



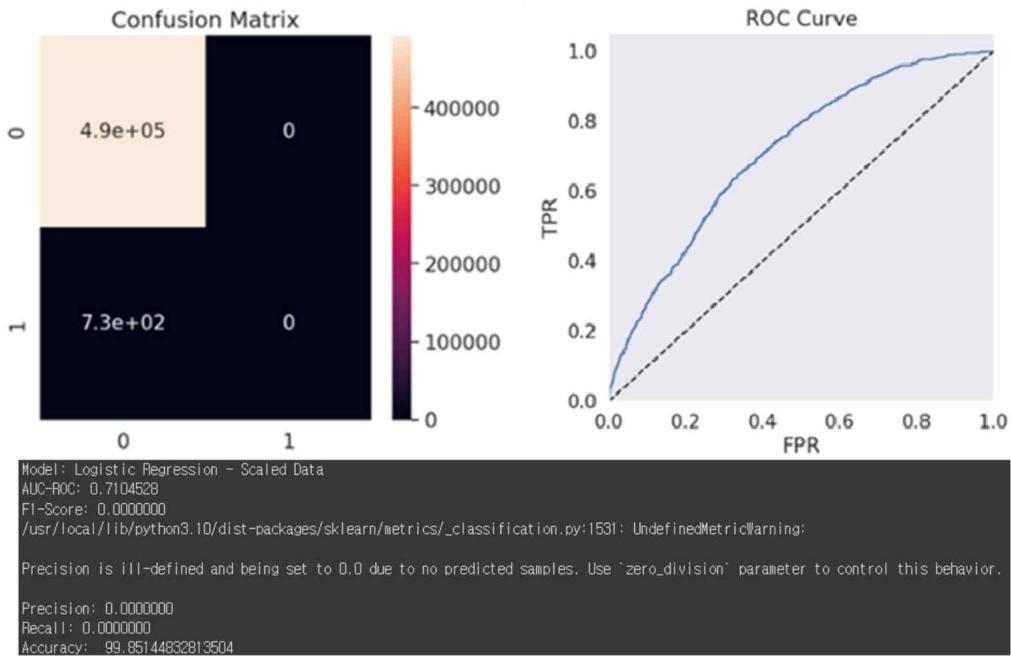
Confusion Matrix를 통해서 정밀도와 재현율을 평가한 결과, XGBoost 모델이 대부분의 정상 거래와 사기 거래를 잘 분류함을 알 수 있다.

AUC(Area Under the Curve) 값이 0.996으로 매우 높게 나타났다. 이 값은 모델이 클래스 간의 분류를 거의 완벽하게 수행함을 의미한다.

- RandomForest 사용 결과



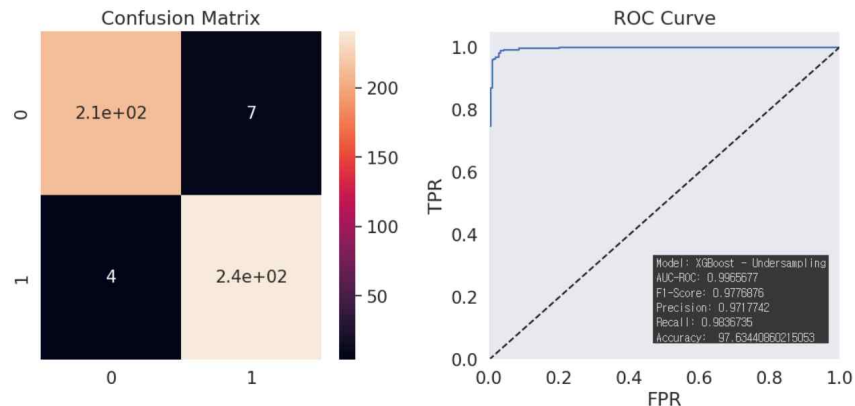
- Logistic Regression 사용 결과



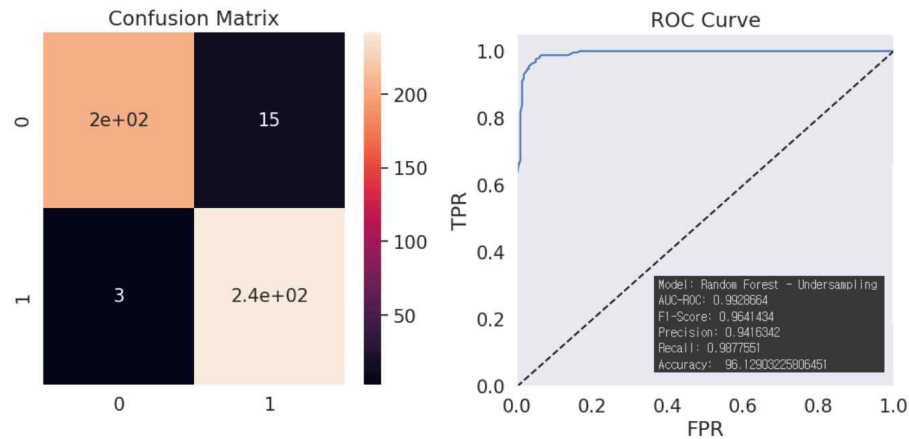
3가지의 모델을 전부 사용해보았을 때, 모델에서 과적합 문제가 발생했다. 이는 train 데이터와 y 값의 비율 불균형이 주된 원인으로 보인다. 데이터가 과적합되어 결과가 치우쳐져 있어, 입력한 데이터만 정확히 맞추게 되는 모델이라 잘못된 모델이 형성되었다. 위 문제를 해결하기 위해 언더샘플링 또는 오버샘플링 기법을 사용해 데이터 비율을 조정하는 것이 필요할 것 같아 다음 단계에서 언더샘플링 기법을 통해서 데이터 비율을 조정하였다.

2) 언더샘플링한 데이터로 진행한 결과

- XGBoost 사용 결과



- RandomForest 사용 결과



기존 데이터에서는 과적합이 발생하여 온전한 탐색을 진행하지 못했으나, 언더샘플링된 데이터를 사용한 경우 사기 거래를 탐지하는 결과를 보여주었다. 이는 불균형한 데이터 문제를 해결하기 위해 언더샘플링이 효과적임을 보여주며, 기존의 모델보다 발전하였음을 알 수 있다.

3) PCA 사용 데이터로 진행한 결과

이후 더 좋은 모델을 위해 PCA를 활용하여 모델을 개발하였다. PCA(Principal Component Analysis)란 고차원 데이터를 저차원으로 축소하는 기법으로, 데이터를 효율적으로 압축하여 주요 정보를 유지하면서, 계산의 복잡성을 줄이고 데이터의 시각화를 가능하게 한다. 이 프로젝트에서는 데이터의 주요 특징을 분석하고 차원 축소를 통해 모델 학습 성능을 높이기 위해 PCA를 사용했다.

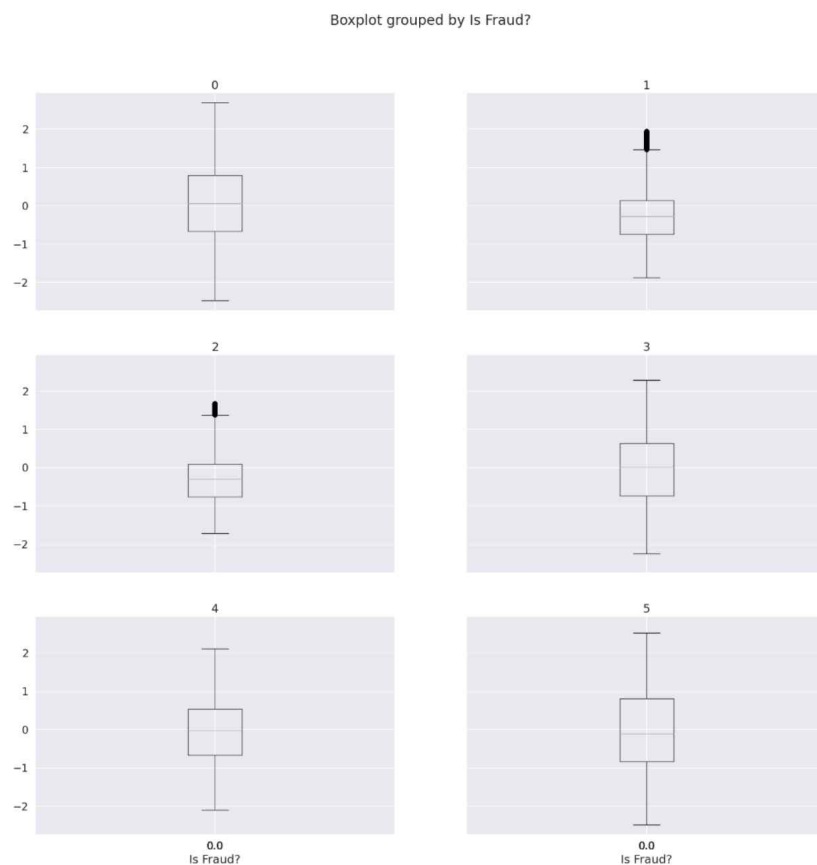
각 변수 간의 분산-공분산 관계를 나타내는 공분산 행렬(Covariance Matrix)을 만들었다. 이를 통해 변수들이 서로 얼마나 상관관계가 있는지, 어떤 변수들이 데이터를 설명하는 데 중요한 역할을 하는지 확인할 수 있다.

```
cov_matrix = np.cov(x.T)
print('Covariance Matrix \n%s' % cov_matrix)
```

Covariance Matrix에서 Eigen Values, Eigen Vectors를 계산했다. 고유값은 각 주성분이 데이터의 분산을 얼마나 잘 설명하는지를 나타내며, 고유 벡터는 새로운 축을 정의한다. 이 축들이 원래의 고차원 데이터를 저차원으로 표현하는 기준이 된다.

```
eig_vals, eig_vecs = np.linalg.eig(cov_matrix)
print('Eigen Vectors \n%s', eig_vecs)
print('\n Eigen Values \n%s', eig_vals)
```

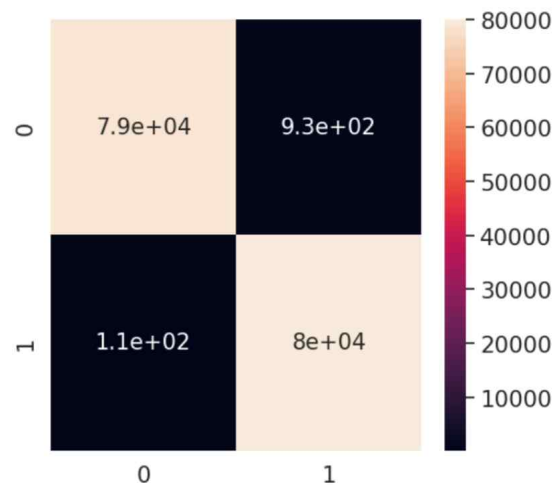
이후 Scikit-learn을 활용하여 PCA를 적용시켰으며 차원 축소된 데이터에서 'Is Fraud?' 별 데이터의 이상치를 탐색하였다.



4) 언더샘플링 + 오버샘플링한 데이터로 진행한 결과

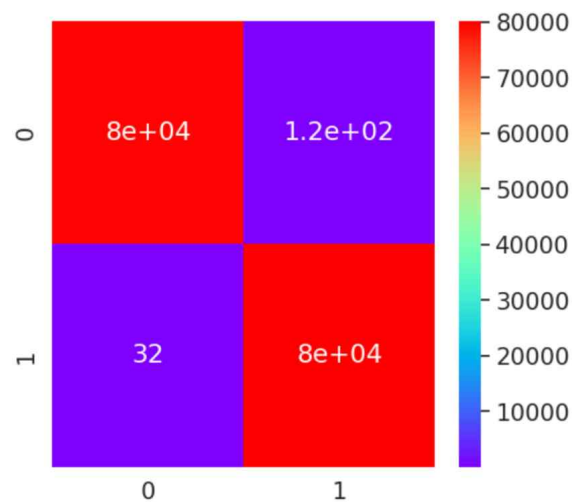
추가로, 사기가 아닌 데이터를 80만개로 언더샘플링하고 사기인 데이터를 80만개로 오버샘플링한 데이터를 모델에 학습하였다.

- XGBoost 사용 결과



Metric	Value
Accuracy	99.34938
Precision	0.98853
Recall	0.99860
F1-score	0.99354
AUC-ROC	0.99968

- RandomForest 사용 결과



Metric	Value
Accuracy	99.90313
Precision	0.99847
Recall	0.99960
F1-score	0.99903
AUC-ROC	0.99997

이전과 유사하게 XGBoost, RandomForestClassifier, LogisticRegression을 학습시킨 결과, RandomForest가 가장 우수한 성능을 보였다.

5) RandomForest + 특정 데이터 집중 학습

이전 결과를 보았을 때 RandomForestClassifier 모델이 가장 적절한 모델로 판단되었다. 또한, 한 번 사기를 당한 사용자가 다시 사기를 당하는 경우가 많다는 데이터의 특징을 발견하였다. 따라서, 해당 모델(RandomForestClassifier)에 특정 데이터 특징을 집중적으로 학습시키는 방법을 실험하였다.

사기를 한 번 당해본 사용자(이하 A라고 한다.)이 다시 사기를 당할 확률이 높다는 것을 전제로 하여 실험하였다. A를 집중적으로 학습하였을 때 결과가 어떻게 달라지는지를 확인하고자 하였다. A는 총 233명으로, 그 중 223명이 다시 사기를 당하였다. 이는 약 95.7%를 차지한다. 또한, 모든 A들의 사기 당한 총 횟수는 2321이고, A들의 사기 당하지 않은 총 횟수는 251075이다. 따라서, A들의 사기를 당한 기록과 사기 당하지 않은 기록 모두를 살펴보고 이를 통해 A에게 가중치를 주어야 한다 예상하였다.

● 1차 시도

첫 번째 시도로 A들의 데이터를 따로 분리하여 학습하였다. 이후 이 데이터를 학습을 시켜 나온 predict_proba(scikit-learn)을 기존의 데이터에 새로운 열로 추가해 다시 학습을 시킨다. 즉, 'Is Fraud?'가 "Yes(코드에서는 1로 치환)"일 확률을 집중적으로 학습시키고, 이후 다시 "No"를 집중적으로 학습시키고자 하였다.

구현1-1 : 데이터(df_fraud)를 'Is Fraud?'의 열을 학습시킬 결과값(Y_train)과 'Is Fraud?' 외의 열인 학습시킬 입력값(X_train)으로 분리하였다. 이를 RandomForestClassifier를 통해 학습을 시키고, 기존의 전체 데이터를 마찬가지로 'Is Fraud' 열을 기준으로 분리하여 테스트하였다. 이 때 나온 predict_proba값을 전체 데이터의 새로운 열로 추가해 주었다.

구현 1-2 : predict_proba 열이 추가된 데이터를 RandomForestClassifier로 다시 학습을 시켰다.

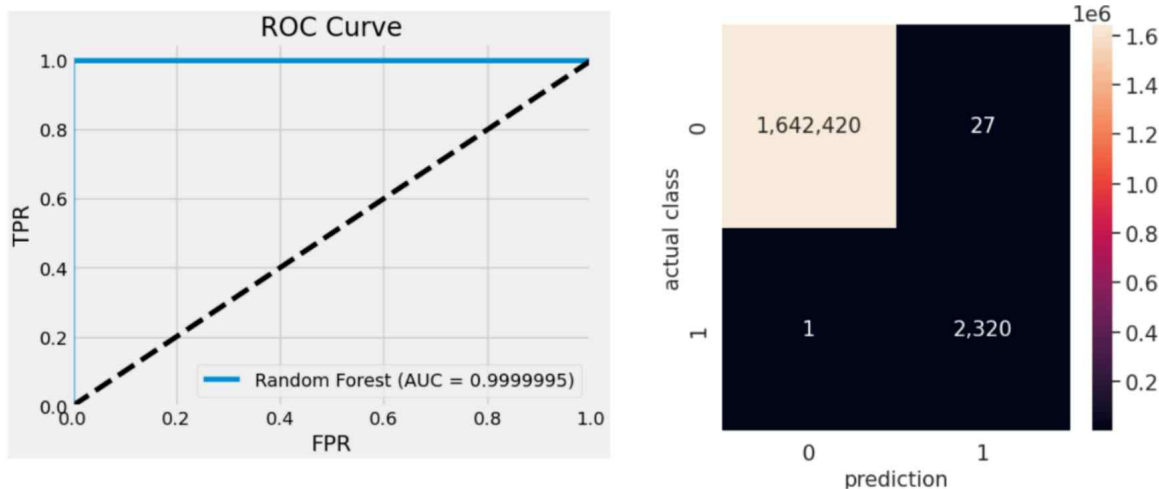
실험 결과 : 과적합이 되는 것을 확인할 수 있었다. 이미 학습한 데이터를 새로운 열로 추가해 다시 학습을 시키는 것은, 이미 학습하여 나온 결과를 다시 학습한 것과 동일하다 판단하였고, 이 때문에 과적합이 발생한 것이라 예상한다. 따라서, 이는 적절한 방법이 아니며 A에 해당하는 상황에 가중치를 주면서 과적합이 일어나지 않는 방법을 생각했다.

● 2차 시도

두 번째 시도로 첫 번째 시도에서 학습시킨 것을 바로 테스트했다. 첫 번째 시도의 구현 1-1에서 A를 학습시킨 데이터는 'Is Fraud?'가 1인 값을 실험1보다 집중적으로 학습을 시키는 방법이다. 그렇다면 A^c인 경우를 따로 학습시키지 않아도 사기당한 경우를 찾아낼 수 있지 않을까 싶었다. 즉 'Is Fraud?'가 0인 경우에 대한 학습은 실험1보다는 약하지만, 'Is Fraud?'가 1인 경우를 찾는 것에 특화되지 않을까 싶었다. 따라서, 첫 번째 시도 구현 1-1에서 한 학습을 통해 전체 데이터를 바로 학습시켰다.

구현2-1 : 데이터(df_fraud)를 'Is Fraud?'의 열을 학습시킬 결과값(Y_train)과 'Is Fraud?' 외의 열인 학습시킬 입력값(X_train)으로 분리하였다. 이를 RandomForestClassifier를 통해 학습을 시키고, 기존의 전체 데이터를 마찬가지로 'Is Fraud' 열을 기준으로 분리하여 테스트하였다.

실험 결과 : 첫 번째 시도에서 과적합이 발생한 것을 줄일 수 있었다. 예상과 동일하게 1을 집중적으로 학습시켜 실제 0인 값을 1로 예측하는 경우가 27건 발생하였고, 실제 1인 값을 0으로 예측하는 경우는 1건 발생하였다. 또한, AUC-ROC값은 0.9999995, F1-Score 값은 0.9940017, Precision은 0.9884960, Recall 은 0.9995692였으며, 최종적인 정확도는 99.99829763224966이다.



실제 0인 값을 1로 예측하는 경우가 실제 1인 값을 0으로 예측하는 경우가 많았다. 따라서, 사기 거래 미탐지 비용보다 정상 거래 오탐지 비용이 작기 때문에 비용적인 부분에서 이득을 얻을 수 있을 것이라 예상한다. 또한, 학습을 하는 데이터를 보다 적게 만들었기 때문에

모델 운영 비용 역시도 비교적 적게 들 것으로 예상된다. 실험을 통해 사기를 당한 경험이 사람들이 다시 사기당하지 않도록 조치를 취해야 할 필요가 있음을 파악할 수 있었다.

4 COST ASSUMPTIONS

창구 송금수수료 (다른 은행으로 보낼 때)	10만원 이하	500원	400원
	10만원 초과 ~ 100만원 이하	2,000원	1,600원
	100만원 초과 ~ 500만원 이하	3,500원	2,800원
	500만원 초과	4,000원	3,200원

```

1 # False Negative - 거래 금액의 100%
2 # False Positive - 기본 고정 비용: 5000원, 국민 은행 타행 이체 수수료 기준 적용
3 # 모델 운영 비용: 검사 건당 10원 -> test한 개수 * 10, 학부 수준의 가벼운 모델이기 때문에 검사 비용은 낮게 책정
4 cost = 0
5 fn_count = 0
6 fn_cost = 0
7 fp_count = 0
8
9 print('False Negative 비용')
10 for i in mismatch_user:
11     # False Negative: 실제로 1인데 모델이 0으로 예측한 경우
12     if mismatches.loc[i]['Actual'] == 1 and mismatches.loc[i]['Predicted'] == 0:
13         fc = abs(data.iloc[i]['Amount'])
14         fn_count += 1
15         fn_cost += fc
16         cost += fc
17         print(f'유저: {i} / 비용: {fc}')
18
19     # False Positive: 실제로 0인데 모델이 1로 예측한 경우
20     elif mismatches.loc[i]['Actual'] == 0 and mismatches.loc[i]['Predicted'] == 1:
21         fp_count += 1
22         cost += 5000 # 기본 고정 비용
23         amount = abs(data.iloc[i]['Amount'])
24
25         # 송금 수수료를 금액 구간에 맞게 추가
26         if amount <= 100000:
27             cost += 500
28         elif amount <= 1000000:
29             cost += 2000
30         elif amount <= 5000000:
31             cost += 3500
32         else:
33             cost += 4000
34
35 cost += 10 * total_tests # 검사 당 10원 비용 추가
36
37 print(f'False Negative {fn_count}회에 대한 비용: {fn_cost}')
38 print(f'False Positive {fp_count}회에 대한 비용: {cost - fn_cost}')
39 print(f'{total_tests}회의 검사에 대하여 총 모델 운영 비용: {cost}원')

```

사기 미탐지 비용

유저: 19005 / 비용: 11600.0
유저: 1852 / 비용: 12500.0
유저: 16223 / 비용: 50300.0
유저: 13190 / 비용: 20500.0
유저: 17394 / 비용: 16800.0
유저: 2226 / 비용: 29400.0
유저: 2047 / 비용: 14200.0
유저: 106 / 비용: 98200.0
유저: 15649 / 비용: 122000.0
유저: 13862 / 비용: 26300.0
유저: 1150 / 비용: 128600.0
유저: 466 / 비용: 72800.0
유저: 14656 / 비용: 1500.0
유저: 13303 / 비용: 77300.0
유저: 110 / 비용: 45500.0
유저: 17348 / 비용: 10600.0

False Negative 16회에 대한 비용: 738100.0

False Positive 28회에 대한 비용: 174000.0

2000회의 검사에 대하여 총 모델 운영 비용: 912100.0원

5 DISCUSSION AND CONCLUSION

진행한 실험 절차를 통해 점진적으로 모델을 향상시킬 수 있었다. 마지막으로 PCA와 이상치 제거 과정을 통해 데이터의 품질을 향상시키고, 모델 학습 과정에서의 노이즈를 최소화할 수 있었다. 모델 개발 및 개선 과정에서 PCA와 이상치 제거가 효과가 있었음을 확인했다. 향후 프로젝트에서도 이상치 탐지 기법이나 자동화된 차원 축소 기법이 데이터 분석의 효율성을 더욱 향상시킬 수 있을 것이라고 예상된다.

이 프로젝트의 결과는 향후 유사한 분류 문제에서 모델을 선정하고 데이터 전처리 기법을 적용하는 데 중요한 지침을 제공할 수 있다. 최종적으로, 데이터의 특성과 모델의 특성을 잘 이해하고, 최적의 전처리 및 튜닝 기법을 적용하는 것이 성능 향상에 결정적인 역할을 한다는 점을 확인하였다.